

Operating Systems and Concurrency

Introduction 3

COMP2007

Dan Marsden

(Chao Chen)

{Chao.Chen,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2026

Recap

Last Lecture

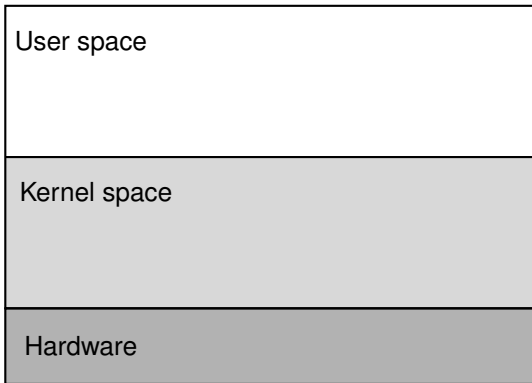
- 1 Operating system **kernel** design closely linked to hardware capabilities
- 2 **Registers** provide data consumed by the CPU
- 3 The **MMU** translates **logical** to **physical addresses** abstracting the details of the memory hardware

Goals for Today

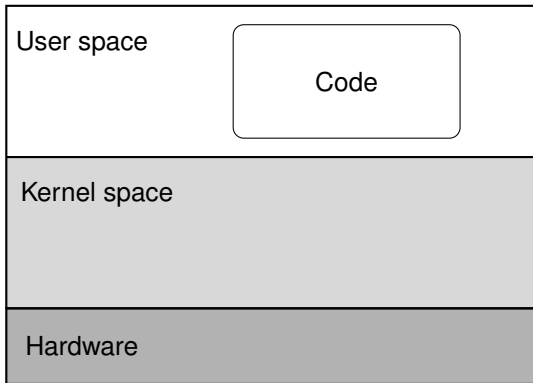
Overview

- **Kernel and user space**
- **Interrupts**
- **System calls**
- **The C programming language** - an operating systems perspective.

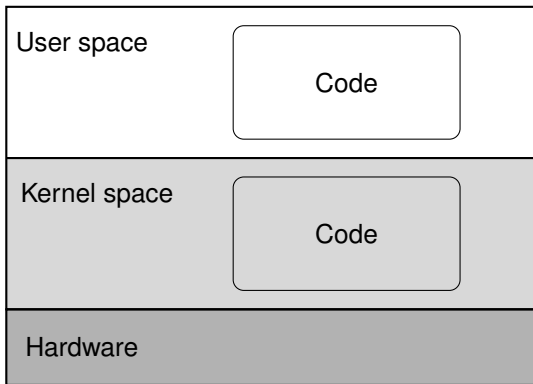
Kernel and user space



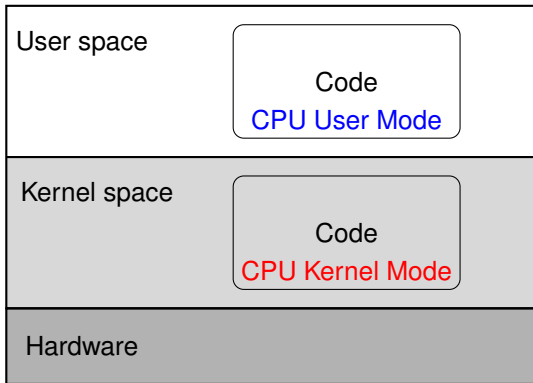
Kernel and user space



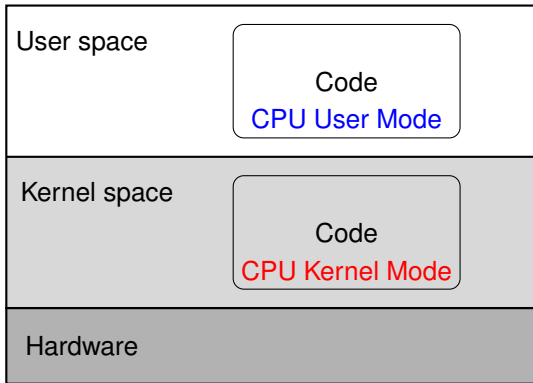
Kernel and user space



Kernel and user space



Kernel and user space



Terminology

The code running in kernel mode is often referred to as the **kernel**. Although this is the core of an operating system, typically they will also have user processes for user interfaces, scheduling daemons, ...

Interrupts

Entering the kernel to respond to events

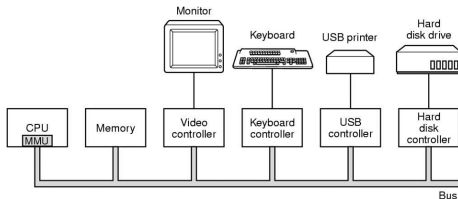


Figure: Simplified computer model (Tanenbaum, 2014)

The CPU cannot usefully live in isolation

Programs, and therefore the CPU, must be able to respond to various events. For example:

- Connected hardware devices might need to communicate
- The passing of time
- Bad things happening - division by zero, hardware faults...

Interrupts

Entering the kernel to respond to events

Interrupts are the mechanism by which such events are handled.

- Abstractly, **an interrupt is a mechanism for changing the normal flow of execution**
- Can happen ^{异步}**asynchronously**, triggered by unpredictable factors external to the CPU, such as user input
 - Sometimes the term **interrupt** is reserved for this class
- Can also happen **synchronously**, triggered directly by the CPU executing an instruction
 - Sometime the term **exception** is used for this class

usually just call interrupt

Interrupts

Entering the kernel to respond to events

Interrupt mechanism (sketch)

Interrupts

Entering the kernel to respond to events

Interrupt mechanism (sketch)

- 1 The CPU is doing some work - for example running a user process doing some calculations

Interrupts

Entering the kernel to respond to events

Interrupt mechanism (sketch)

- 1 The CPU is doing some work - for example running a user process doing some calculations
- 2 An interrupt is signalled by a hardware device - for example to indicate some IO data is available

Interrupts

Entering the kernel to respond to events

Interrupt mechanism (sketch)

- 1 The CPU is doing some work - for example running a user process doing some calculations
- 2 An interrupt is signalled by a hardware device - for example to indicate some IO data is available
- 3 The CPU records aspects of its current state, **switches to kernel mode**, and runs code in a **handler** to service the interrupt

Interrupts

Entering the kernel to respond to events

Interrupt mechanism (sketch)

- 1 The CPU is doing some work - for example running a user process doing some calculations
- 2 An interrupt is signalled by a hardware device - for example to indicate some IO data is available
- 3 The CPU records aspects of its current state, switches to kernel mode, and runs code in a handler to service the interrupt
- 4 Once completed, the CPU is returned to processing other tasks

Interrupts

Entering the kernel to respond to events

Challenges

Handling interrupts is one of the more challenging tasks of operating system implementation.

Interrupts

Entering the kernel to respond to events

Challenges

Handling interrupts is one of the more challenging tasks of operating system implementation.

- Interrupts can come at any time
 - Ideally handling interrupts should not take long
 - Handlers may split work into a **top** component dealt with immediately, and a **bottom** component scheduled to be dealt with later

Interrupts

Entering the kernel to respond to events

Challenges

Handling interrupts is one of the more challenging tasks of operating system implementation.

- Interrupts can come at any time
 - Ideally handling interrupts should not take long
 - Handlers may split work into a **top** component dealt with immediately, and a **bottom** component scheduled to be dealt with later
- Interrupts may be interrupted by other interrupts - it must be possible to **nest** interrupt handlers

Interrupts

Entering the kernel to respond to events

Challenges

Handling interrupts is one of the more challenging tasks of operating system implementation.

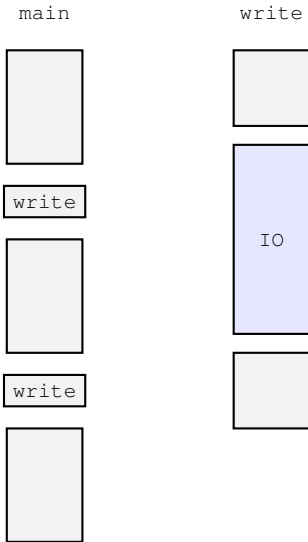
- Interrupts can come at any time
 - Ideally handling interrupts should not take long
 - Handlers may split work into a **top** component dealt with immediately, and a **bottom** component scheduled to be dealt with later
- Interrupts may be interrupted by other interrupts - it must be possible to 嵌套 nest interrupt handlers
- Sometimes critical code cannot be interrupted and they must be temporarily disabled

Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

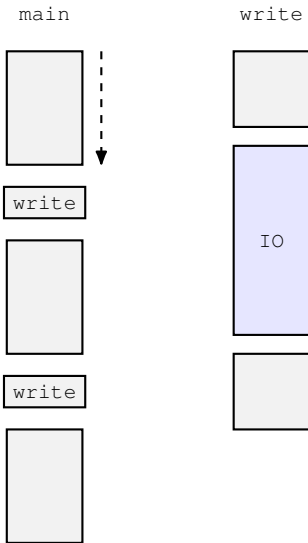


Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

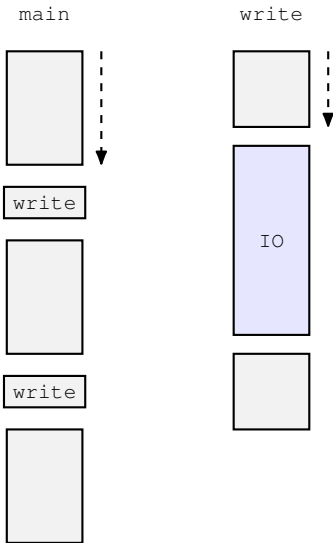


Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

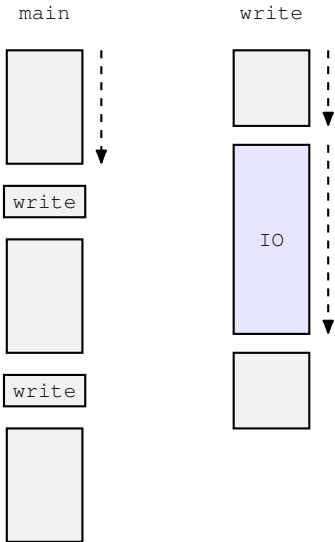


Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

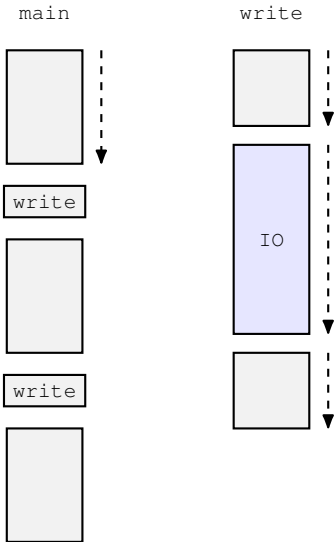


Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

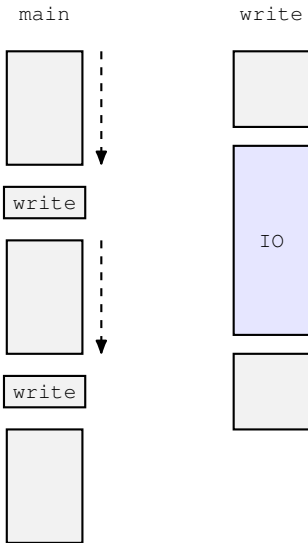


Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

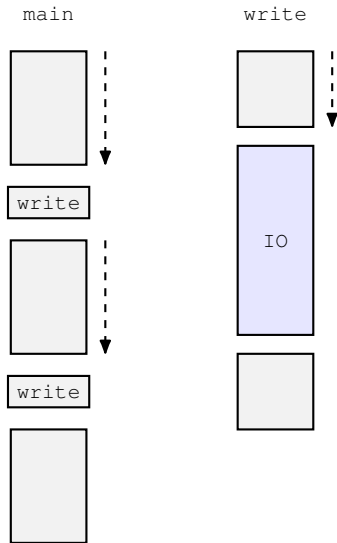


Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

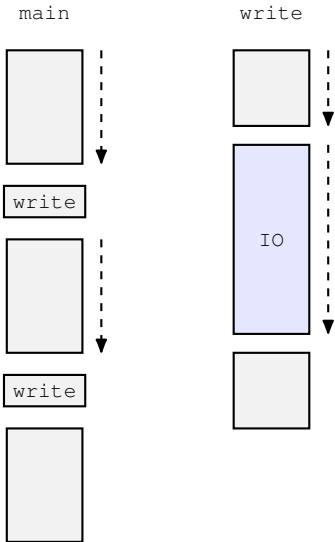


Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

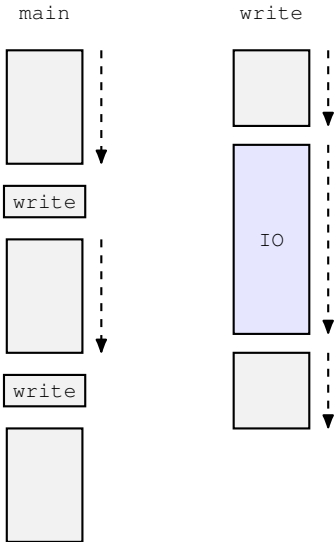


Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

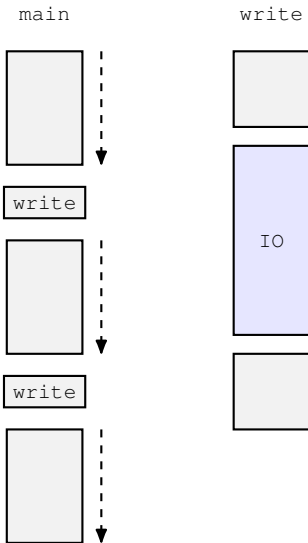


Interrupts

CPU utilisation

Blocking IO

Naive IO which wastes CPU cycles waiting for slow devices to respond.

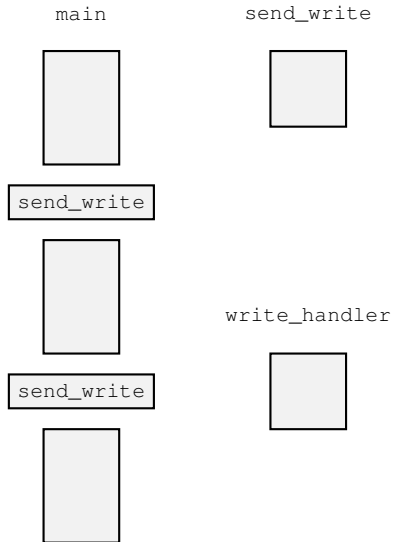


Interrupts

CPU utilisation

Interrupts

Better CPU usage supported by interrupts “calling back” when there is work to do.

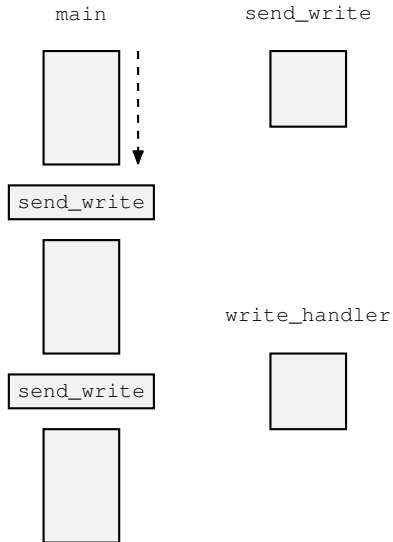


Interrupts

CPU utilisation

Interrupts

Better CPU usage supported by interrupts “calling back” when there is work to do.

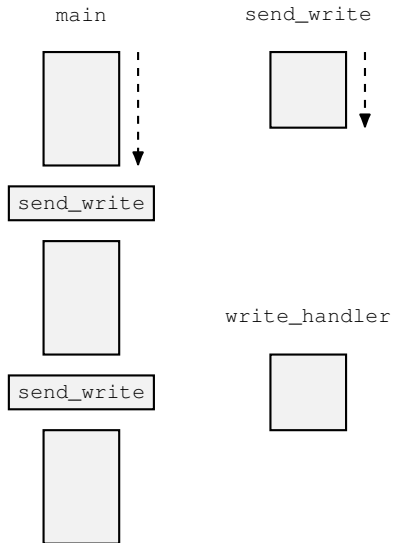


Interrupts

CPU utilisation

Interrupts

Better CPU usage supported by interrupts “calling back” when there is work to do.

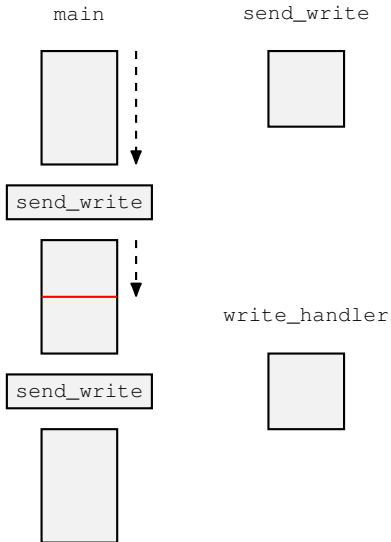


Interrupts

CPU utilisation

Interrupts

Better CPU usage supported by interrupts “calling back” when there is work to do.

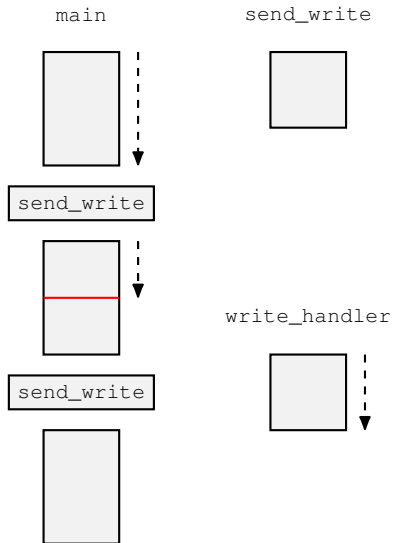


Interrupts

CPU utilisation

Interrupts

Better CPU usage supported by interrupts “calling back” when there is work to do.

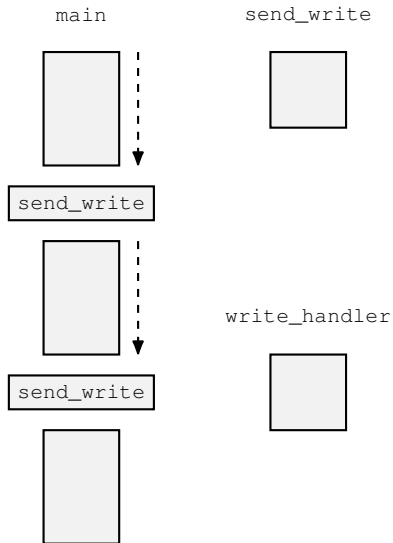


Interrupts

CPU utilisation

Interrupts

Better CPU usage supported by interrupts “calling back” when there is work to do.

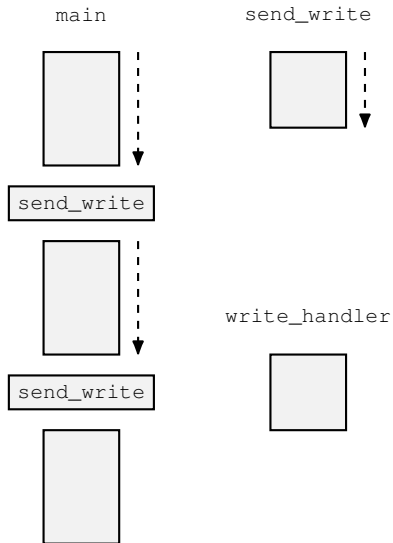


Interrupts

CPU utilisation

Interrupts

Better CPU usage supported by interrupts “calling back” when there is work to do.

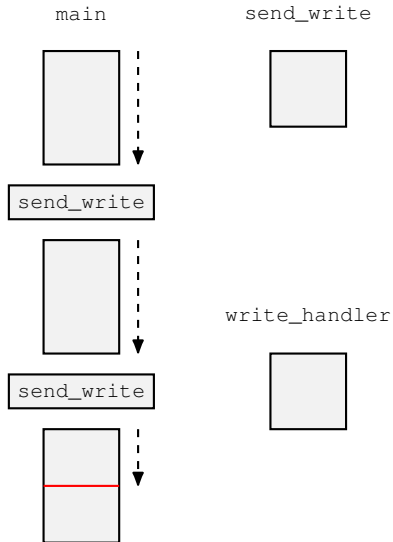


Interrupts

CPU utilisation

Interrupts

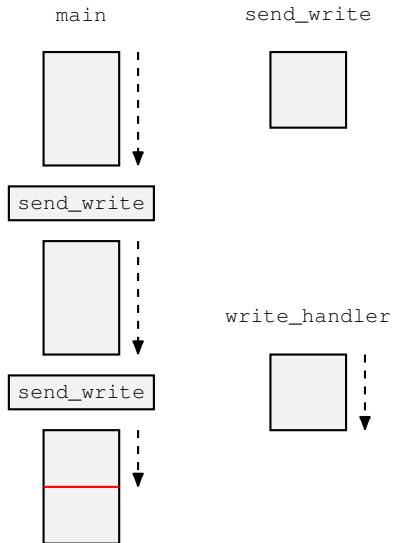
Better CPU usage supported by interrupts “calling back” when there is work to do.



CPU utilisation

Interrupts

Better CPU usage supported
by interrupts “calling back”
when there is work to do.



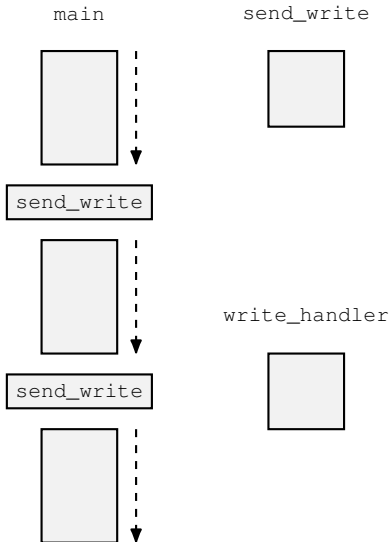
Interrupts

CPU utilisation

Interrupts

Better CPU usage supported by interrupts “calling back” when there is work to do.

Blocking I/O wastes CPU cycles by making the CPU wait for slow devices, whereas interrupts allow the CPU to continue useful work and invoke an interrupt handler only when the device requires attention.



System calls

Entering the kernel by software request

System calls are how programs request services from the operating system.

Example (Typical system calls)

- Requesting memory
- Accessing files
- Running programs (processes)
- Accessing concurrency features

System calls

Entering the kernel by software request

We can distinguish between an API and systems calls.

- An API is a programming interface - typically a **library of functions that run in user space**, for example pthreads
- To provide the required features **an API may need to make system calls to access the required functionality**
- The relationship is *not* necessarily one-to-one - **a single API function may invoke zero, one or many system calls**
- Often programmers use less fussy terminology, and simply refer to the API functions as system calls

System calls

Entering the kernel by software request

Question

How can a system call allow a user process to run kernel space code?

A system call uses a trap instruction to generate a synchronous interrupt, causing the CPU to switch from user mode to kernel mode and execute a predefined kernel system-call handler. After the requested service is completed, control returns to the user process.

System calls

Entering the kernel by software request

System call mechanisms (naive sketch)

System calls

Entering the kernel by software request

System call mechanisms (naive sketch)

- 1 Some user code is running

System calls

Entering the kernel by software request

System call mechanisms (naive sketch)

- 1 Some user code is running
- 2 A system call is required - each has a **unique system call number**

System calls

Entering the kernel by software request

System call mechanisms (naive sketch)

- 1 Some user code is running
- 2 A system call is required - each has a **unique system call number**
- 3 The system call number is stored in a designated register

System calls

Entering the kernel by software request

System call mechanisms (naive sketch)

- 1 Some user code is running
- 2 A system call is required - each has a **unique system call number**
- 3 The system call number is stored in a designated register
- 4 The system call parameters are stored in designated registers

System calls

Entering the kernel by software request

System call mechanisms (naive sketch)

- 1 Some user code is running
- 2 A system call is required - each has a **unique system call number**
- 3 The system call number is stored in a designated register
- 4 The system call parameters are stored in designated registers
- 5 A synchronous interrupt (exception) is triggered by an instruction referred to as a **trap**

System calls

Entering the kernel by software request

System call mechanisms (naive sketch)

- 1 Some user code is running
- 2 A system call is required - each has a **unique system call number**
- 3 The system call number is stored in a designated register
- 4 The system call parameters are stored in designated registers
- 5 A synchronous interrupt (exception) is triggered by an instruction referred to as a **trap**
- 6 The interrupt is handled by kernel mode code, which calls a **system call service routine** which delivers the required functionality

System calls

Entering the kernel by software request

System call mechanisms (naive sketch)

- 1 Some user code is running
- 2 A system call is required - each has a **unique system call number**
- 3 The system call number is stored in a designated register
- 4 The system call parameters are stored in designated registers
- 5 A synchronous interrupt (exception) is triggered by an instruction referred to as a trap
- 6 The interrupt is handled by kernel mode code, which calls a **system call service routine** which delivers the required functionality
- 7 The operating system continues running the calling code

System calls

Entering the kernel by software request

System call mechanisms (slightly less naive)

- The system call service routine **may not actually service the request immediately** - for example if it must wait on some resource such as IO
- The operating system **may not continue running the original caller**
- **System calls are the kernel's big chance to get work done for the benefit of everything running on the system**

System calls

Entering the kernel by software request

System call mechanisms (fussier details)

- Details vary by operating system
- Modern approaches may avoid interrupt based mechanisms and use other CPU support for efficiency reasons

The C Programming Language

Criteria for an OS Implementation Language

Question

Why would we choose such an old language like C for OS implementation?

```
1  #include <stdio.h>
2
3  int main() {
4      printf("Hello Operating Systems and Concurrency\n");
5  }
```

The C Programming Language

Criteria for an OS Implementation Language

Performance

As general purpose software, an OS must be fast enough for as many users as possible.

The C Programming Language

Criteria for an OS Implementation Language

Performance

As general purpose software, an OS must be fast enough for as many users as possible.

- A slow OS slows every program that it runs.

The C Programming Language

Criteria for an OS Implementation Language

Performance

As general purpose software, an OS must be fast enough for as many users as possible.

- A slow OS slows every program that it runs.
- Mobile hardware must also consider energy efficiency.

The C Programming Language

Criteria for an OS Implementation Language

Performance

As general purpose software, an OS must be fast enough for as many users as possible.

- A slow OS slows every program that it runs.
- Mobile hardware must also consider energy efficiency.
- Bias toward performance over simplicity, elegance and maintainability.

The C Programming Language

Criteria for an OS Implementation Language

Performance

As general purpose software, an OS must be fast enough for as many users as possible.

- A slow OS slows every program that it runs.
- Mobile hardware must also consider energy efficiency.
- Bias toward performance over simplicity, elegance and maintainability.
- Cannot sacrifice correctness.

The C Programming Language

Criteria for an OS Implementation Language

Portability

As OS development is difficult, reuse is desirable.

The C Programming Language

Criteria for an OS Implementation Language

Portability

As OS development is difficult, reuse is desirable.

- Must run directly on the hardware - no interpreted languages.

The C Programming Language

Criteria for an OS Implementation Language

Portability

As OS development is difficult, reuse is desirable.

- Must run directly on the hardware - no interpreted languages.
- Ideally must be able to compile for many different hardware platforms.

The C Programming Language

Criteria for an OS Implementation Language

Predictability

The behaviour of an OS must be predictable for user programs.

The C Programming Language

Criteria for an OS Implementation Language

Predictability

The behaviour of an OS must be predictable for user programs.

- Example - games cannot tolerate unpredictable delays slowing frame rates and responsiveness.

The C Programming Language

Criteria for an OS Implementation Language

Predictability

The behaviour of an OS must be predictable for user programs.

- Example - games cannot tolerate unpredictable delays slowing frame rates and responsiveness.
- Unpredictable behaviour such as garbage collection is inappropriate.

The C Programming Language

Criteria for an OS Implementation Language

Summary - The three P's

An operating system implementation language should be:

- **Performant**
- **Portable**
- **Predictable**

Test your understanding

Small kernel: simpler, safer, easier to maintain, but may have more overhead.

Large kernel: faster and more feature-rich, but more complex and risky.

- What would be the advantages and disadvantages of a small kernel with a limited collection of system calls?
- What would be the advantages and disadvantages of a large kernel with a rich collection of system calls?
- How much harm can a bug in a user process cause?
- How much harm can a bug in the kernel cause?

Bug in user process: usually only crashes that process.

Bug in kernel: may crash or damage the whole system.

Introduction 3 - Core Revision Summary

Kernel/user space - interrupts - CPU utilisation - system calls - C for OS implementation

1. Topic Overview

This lecture explains how an operating system safely moves between user-mode execution and privileged kernel-mode services. Hardware events enter the kernel through interrupts; software requests enter through system calls. It also compares blocking I/O with interrupt-driven I/O and explains why C is suitable for OS implementation.

Lecture flow: user/kernel separation -> interrupts -> blocking I/O vs interrupt-driven I/O -> system calls and traps -> API vs system call -> C and the three P's.

2. Core Concepts

Kernel and user space

- Code running in CPU kernel mode is commonly called the kernel. User programs normally run in user mode with restricted privileges.
- The separation protects the system because user code cannot freely access hardware or arbitrary kernel memory.

Interrupts

- An interrupt changes the normal flow of execution.
- **Asynchronous interrupt:** triggered by an unpredictable external event, such as hardware or user input.
- **Synchronous interrupt / exception:** triggered directly by CPU instruction execution.
- Typical path: save relevant CPU state -> switch to kernel mode -> run an interrupt handler -> continue other processing.
- Handlers should be short. Work can be split into an immediate top part and a deferred bottom part.
- Interrupt handlers may be nested. Critical code may temporarily disable interrupts.

CPU utilisation

- **Blocking I/O:** naive waiting for a slow device wastes CPU cycles.
- **Interrupt-driven I/O:** the CPU continues useful work; the device later signals an interrupt when attention is needed.

System calls

- System calls let programs request OS services such as memory, files, processes, and concurrency features.
- **API is not the same as a system call.** An API is a programming interface, usually in user space; one API function may make zero, one, or many system calls.
- In the lecture model, a trap causes a synchronous exception that switches execution into kernel mode.

C for OS implementation

- The three P's: **Performant, Portable, Predictable.**
- Performance matters system-wide; portability enables reuse across platforms; predictability avoids unexpected delays such as garbage collection.

3. Key Mechanisms / Algorithms

Interrupt handling path

User code / current task -> interrupt occurs -> save state -> switch to kernel mode -> handler runs -> return to useful processing

Blocking I/O vs interrupt-driven I/O

- Blocking I/O: caller waits while the device works, so CPU time can be wasted.
- Interrupt-driven I/O: start the device operation, keep the CPU busy elsewhere, then run a handler when the device signals completion or requests service.

System call path (naive sketch)

1. User code is running.
2. A system call is required; each has a unique system-call number.
3. The system-call number is placed in a designated register.
4. Parameters are placed in designated registers.
5. A trap triggers a synchronous interrupt / exception.
6. Kernel-mode code handles it and calls the required system-call service routine.
7. The OS later continues execution.

Less naive model

- The service routine may not complete immediately, for example while waiting for I/O.
- The OS may not continue running the original caller immediately.
- A system call gives the kernel an important opportunity to schedule and manage resources for the benefit of the whole system.

API relationship

Application code -> API/library function in user space -> zero/one/many system calls -> kernel service

4. Rules / Conditions

Interrupt rules

- Asynchronous = external/unpredictable event; synchronous = caused by instruction execution.
- Keep handlers short; defer non-urgent work.
- Nested interrupts may occur; critical regions may temporarily disable interrupts.

System-call rules

- User code cannot jump to arbitrary kernel code; entry must use a controlled OS/hardware mechanism.
- The lecture model uses a system-call number and parameters in registers.
- API calls and system calls are not one-to-one.
- Real mechanisms vary by OS; modern CPUs may provide faster entry mechanisms than interrupt-style traps.

The three P's

Performant: a slow OS slows all programs.
Portable: should compile for multiple hardware platforms and run directly on hardware.
Predictable: avoid unexpected pauses that hurt responsiveness.

5. Comparison & Common Confusions

Concept A	Concept B	Key difference / revision point
User space	Kernel space	Restricted user privileges vs privileged OS code with access to system resources.
Asynchronous interrupt	Synchronous exception	External/unpredictable event vs event caused directly by instruction execution.
Blocking I/O	Interrupt-driven I/O	Waiting wastes CPU time; interrupts allow useful work until the device needs attention.
API	System call	Programming interface in user space vs controlled request for kernel functionality; mapping is not 1:1.
Small kernel	Large kernel	Small: simpler/safer/easier to maintain but may add overhead. Large: richer/faster but more complex and risky.
User-process bug	Kernel bug	User bug usually harms one process; kernel bug may affect or crash the entire system.

6. Exam / Revision Focus

Must Know

- Definitions: kernel/user space, interrupt, handler, asynchronous, synchronous/exception, system call, trap, API, blocking I/O.
- Interrupt path: save state -> kernel mode -> handler -> return.
- System-call path: call number + parameters -> controlled kernel entry -> service routine.
- The three P's: Performant, Portable, Predictable.

Must Understand

- Why interrupt-driven I/O improves CPU utilisation compared with blocking I/O.
- Why a system call can block and why the OS may run another process instead of immediately resuming the caller.
- Why system calls are a controlled gateway from user mode to kernel mode.
- Why putting more functionality in the kernel can improve directness/performance but increases complexity and bug impact.

Easy to Confuse

- Interrupt is the broad mechanism; exception is the lecture term for the synchronous class.
- API call does not mean system call.
- Blocking I/O is waiting behaviour; interrupts are a mechanism that can reduce wasted CPU time.
- A system call does not guarantee immediate completion or immediate return to the original caller.
- The trap mechanism shown is a simplified teaching model; real OS details vary.